Ex.No:8

Experiment : Write a Python Program to Implement Depth-First Search (DFS)

Aim

The aim of this experiment is to write a Python program to implement the Depth-First Search (DFS) algorithm to traverse all the vertices of a graph.

Objectives

- To understand the DFS algorithm.
- To write a Python program for DFS.
- To traverse all vertices of a graph.
- To display the DFS traversal order.

Algorithm

1. Start.
2. Create a graph using an adjacency list.
3. Select the starting vertex.
4. Mark the starting vertex as visited.
5. Display the current vertex.
6. Visit each unvisited adjacent vertex recursively.
7. Repeat until all vertices are visited.
8. Stop.

Program (Python)

```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': ['F'],
   'F': []
}
```

```
visited = set()
def dfs(vertex):
    if vertex not in visited:
        print(vertex, end=" ")
        visited.add(vertex)
        for neighbor in graph[vertex]:
            dfs(neighbor)
print("DFS Traversal:")
dfs('A')
```

Sample Input

Graph:

A → B, C

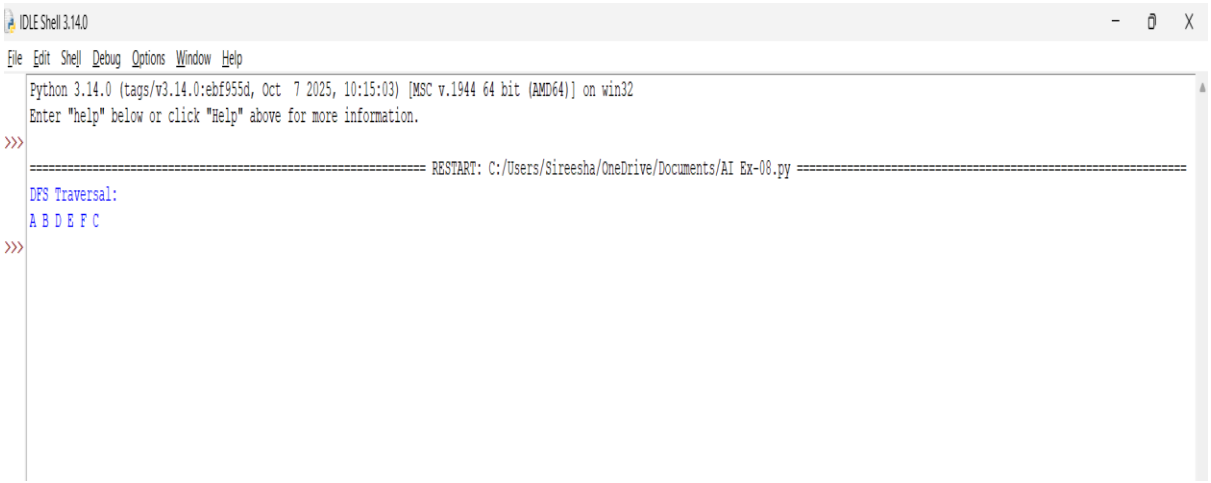
B → D, E

C → F

E → F

Starting Vertex: A

Output



Result

The Python program to implement the Depth-First Search (DFS) algorithm was successfully executed. The graph was traversed using the depth-first approach, and the DFS traversal order was displayed correctly.

Ex.No-9

Experiment : Write a Python Program to Implement the Travelling Salesman Problem (TSP)

Aim

The aim of this experiment is to write a Python program to solve the Travelling Salesman Problem by finding the shortest route that visits each city exactly once and returns to the starting city.

Objectives

- To understand the Travelling Salesman Problem (TSP).
- To write a Python program to solve TSP.
- To find the minimum travel cost.
- To display the shortest possible route.

Algorithm

1. Start.
2. Define the distance matrix between cities.
3. Select the starting city.
4. Generate all possible routes.
5. Calculate the total distance of each route.
6. Find the route with the minimum distance.
7. Display the shortest route and its cost.
8. Stop.

Program (Python)

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
```

```
cities = [0, 1, 2, 3]
start = 0
min_cost = float('inf')
best_path = None
for path in permutations(cities[1:]):
    route = (start,) + path + (start,)
    cost = 0
    for i in range(len(route) - 1):
        cost += graph[route[i]][route[i + 1]]
    if cost < min_cost:
        min_cost = cost
        best_path = route
print("Shortest Route:", best_path)
print("Minimum Cost:", min_cost)
```

Sample Input

Distance Matrix

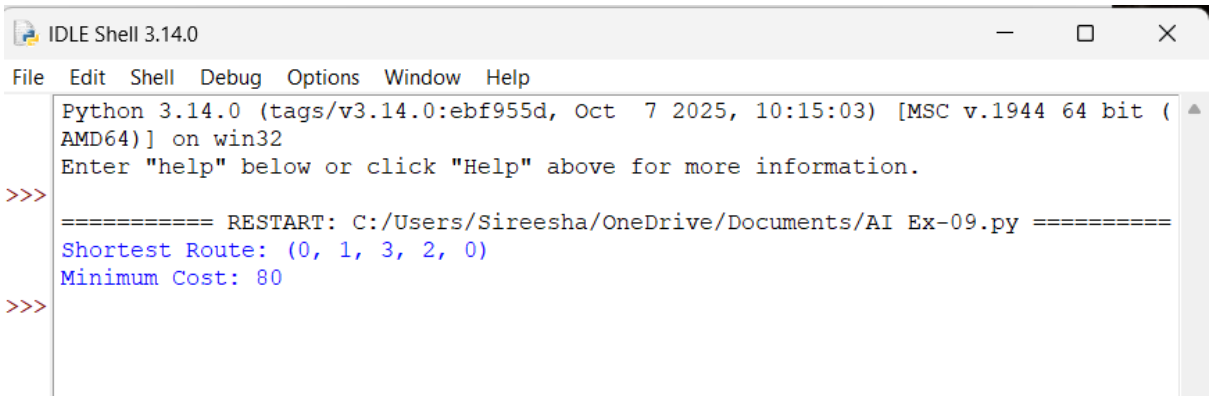
0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output



Result

The Python program for the Travelling Salesman Problem (TSP) was successfully implemented. The program found the shortest route that visits all cities exactly once and returns to the starting city, and it displayed the minimum travel cost correctly.

Ex.No-10

Experiment 10: Write a Python Program to Implement the A Algorithm*

Aim

The aim of this experiment is to write a Python program to implement the A* algorithm to find the shortest path between a start node and a goal node.

Objectives

- To understand the A* search algorithm.
- To write a Python program for A*.
- To find the shortest path between two nodes.
- To display the shortest path.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Select the start node and goal node.
4. Add the start node to the open list.
5. Select the node with the lowest cost.
6. Explore its neighboring nodes and update their costs.
7. Repeat until the goal node is reached.
8. Display the shortest path.
9. Stop.

Program (Python)

```
graph = {
  'A': {'B': 1, 'C': 3},
  'B': {'D': 3, 'E': 6},
  'C': {'F': 5},
```

```
    'D': {'G': 2},
    'E': {'G': 1},
    'F': {'G': 2},
    'G': {}
}
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 2,
    'E': 1,
    'F': 2,
    'G': 0
}
def a_star(start, goal):
    open_list = {start}
    closed_list = set()
    g = {start: 0}
    parent = {start: start}
    while open_list:
        node = min(open_list, key=lambda x: g[x] + heuristic[x])
        if node == goal:
            path = []
            while parent[node] != node:
                path.append(node)
                node = parent[node]
            path.append(start)
            path.reverse()
            return path
```

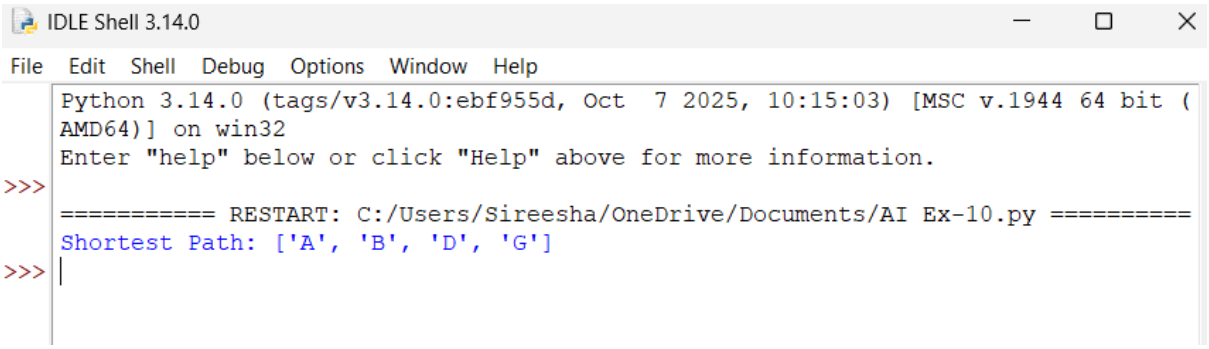
```
        open_list.remove(node)
        closed_list.add(node)
        for neighbor, cost in graph[node].items():
            if neighbor in closed_list:
                continue
            new_cost = g[node] + cost
            if neighbor not in open_list or new_cost < g.get(neighbor, float('inf')):
                g[neighbor] = new_cost
                parent[neighbor] = node
                open_list.add(neighbor)
    return None
path = a_star('A', 'G')
print("Shortest Path:", path)
```

Sample Input

Start Node = A

Goal Node = G

Sample Output



Result

The Python program for the A* (A-Star) Algorithm was successfully implemented. The program found the shortest path from the start node to the goal node using the heuristic function and displayed the optimal path correctly.

Ex.No-11

Experiment : Write a Python Program for Map Coloring to Implement CSP

Aim

The aim of this experiment is to write a Python program to solve the Map Coloring problem using Constraint Satisfaction Problem (CSP) techniques by assigning different colors to adjacent regions.

Objectives

- To understand the Map Coloring problem.
- To implement CSP using Python.
- To assign different colors to adjacent regions.
- To display the valid color assignment.

Algorithm

1. Start.
2. Define the map with its neighboring regions.
3. Define the available colors.
4. Assign a color to each region.
5. Check that no two adjacent regions have the same color.
6. If a conflict occurs, try another color.
7. Repeat until all regions are colored.
8. Display the color assigned to each region.
9. Stop.

Program (Python)

```
# Map Coloring using CSP (Backtracking)

regions = ['A', 'B', 'C', 'D']

neighbors = {

   'A': ['B', 'C'],

   'B': ['A', 'C', 'D'],

   'C': ['A', 'B', 'D'],

   'D': ['B', 'C']
```

```
}
colors = ['Red', 'Green', 'Blue']
assignment = {}
def is_safe(region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True
def solve(index):
    if index == len(regions):
        return True
    region = regions[index]
    for color in colors:
        if is_safe(region, color):
            assignment[region] = color
            if solve(index + 1):
                return True
            del assignment[region]
    return False
if solve(0):
    print("Color Assignment:")
    for region in regions:
        print(region, "->", assignment[region])
else:
    print("No Solution Exists")
```

Sample Input

Regions: A, B, C, D

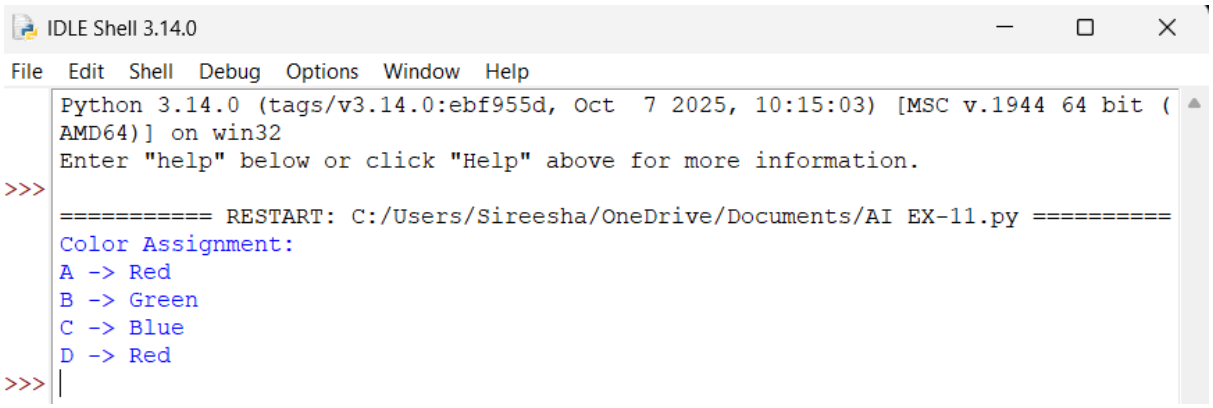
Colors:

Red

Green

Blue

Sample Output



Result

The Python program for the Map Coloring Problem using Constraint Satisfaction Problem (CSP) was successfully implemented. The program assigned different colors to adjacent regions without any conflicts and displayed a valid color assignment.

Ex.No-12

Experiment : Write a Python Program for the Tic-Tac-Toe Game

Aim

The aim of this experiment is to write a Python program to implement the Tic-Tac-Toe game, where two players take turns marking **X** and **O** on a 3×3 board until one player wins or the game ends in a draw.

Objectives

- To understand the Tic-Tac-Toe game.
- To write a Python program for the game.
- To allow two players to play alternately.
- To determine the winner or declare a draw.

Algorithm

1. Start.
2. Create an empty 3×3 game board.
3. Allow Player X and Player O to take turns.
4. Accept the row and column as input.

5. Place the player's symbol in the selected position.
6. Check if any player has won.
7. If all cells are filled and no player wins, declare a draw.
8. Stop.

Program (Python)

```
board = [' ' for _ in range(9)]
def print_board():
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
def check_win(player):
    wins = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ]
    for w in wins:
        if board[w[0]] == board[w[1]] == board[w[2]] == player:
            return True
    return False
player = 'X'
for i in range(9):
    print_board()
    pos = int(input("Enter position (1-9): ")) - 1
    if board[pos] == ' ':
        board[pos] = player
    else:
```

```
            print("Position already filled!")
            continue
        if check_win(player):
            print_board()
            print(player, "wins!")
            break
        player = 'O' if player == 'X' else 'X'
    else:
        print_board()
        print("Match Draw!")
```

Sample Input

Player X: 1

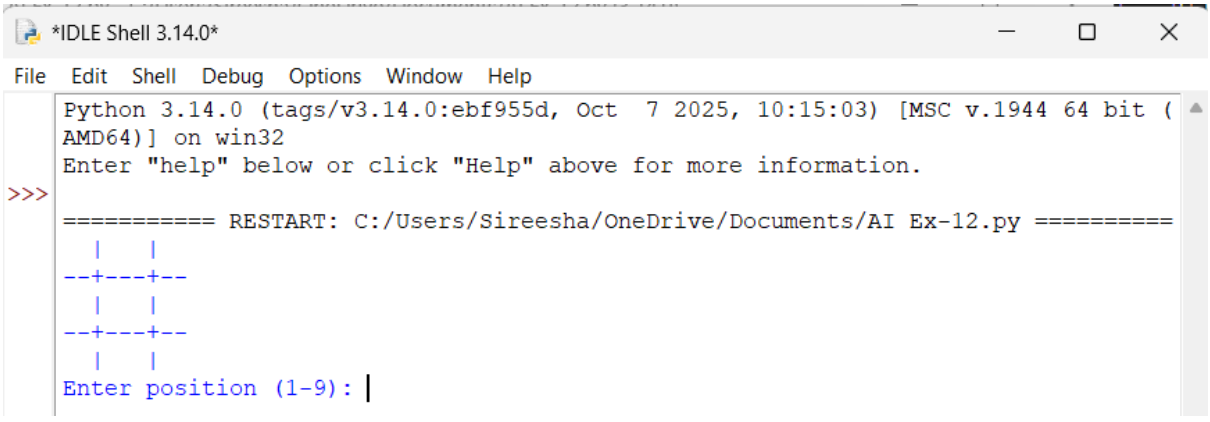
Player O: 5

Player X: 2

Player O: 8

Player X: 3

Sample Output



Result

The Python program for the Tic-Tac-Toe Game was successfully implemented. The program allowed two players to play alternately, checked for a winner after each move, and correctly displayed the winner or declared the game as a draw.